ระบบบริหารการจองพื้นที่และเข้าใช้สำนักงาน

กฤษณะ พิพัฒน์พฤติกุล*[1] และ สุรีย์ ฟูนิลกุล[2]

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี 126 ถนนประชาอุทิศ แขวงบางมด เขตทุ่งครุ กรุงเทพฯ 10140

บทคัดย่อ

บทความนี้นำเสนอการพัฒนาระบบบริหารการจองพื้นที่และเข้าใช้สำนักงาน หรือ CoDesk เพื่อสนับสนุนการบริหารจัดการการเข้าใช้ออฟฟิศขององค์กรให้มีความเป็นระบบมากขึ้น จากปัญหาการจัดการข้อมูลการจองผ่านหลายช่องทาง ซึ่งอาจก่อให้เกิดการจองซ้ำ ความคลาดเคลื่อนของข้อมูล การควบคุมจำนวนผู้เข้าใช้งานไม่สอดคล้องกับความจุของแต่ละฝ่าย และการเข้าถึงข้อมูลเกินขอบเขตของผู้ใช้งานแต่ละกลุ่ม ระบบถูกพัฒนาในรูปแบบ Web Application โดยใช้ React และ TypeScript สำหรับส่วนติดต่อผู้ใช้งาน ใช้ ASP.NET สำหรับประมวลผลตรรกะการทำงานของระบบ และใช้ Supabase PostgreSQL เป็นฐานข้อมูลหลักสำหรับจัดเก็บข้อมูลเชิงสัมพันธ์และสนับสนุนการควบคุมการเข้าถึงข้อมูล จากผลการพัฒนาพบว่า ระบบสามารถรองรับการสร้าง แก้ไข และยกเลิกรายการจอง ตรวจสอบและป้องกันการจองซ้ำ แจ้งเตือนเมื่อวันที่จองตรงกับวันหยุด ควบคุมจำนวนผู้จองตามความจุของแต่ละฝ่าย และกำหนดการมองเห็นข้อมูลตามบทบาทและฝ่ายงานของผู้ใช้งาน นอกจากนี้ ข้อมูลการจองยังสามารถนำมาสรุปเป็นรายงานเพื่อใช้ตรวจสอบจำนวนผู้จองและการใช้ความจุของแต่ละฝ่ายได้ ส่งผลให้ข้อมูลการเข้าใช้ออฟฟิศถูกจัดเก็บและตรวจสอบภายใต้กติกาเดียวกัน ลดความคลาดเคลื่อนในการจัดการข้อมูล และสนับสนุนการนำข้อมูลไปใช้ประกอบการบริหารจัดการพื้นที่สำนักงานได้อย่างเป็นระบบมากขึ้น

คำสำคัญ: การควบคุมความจุ, การควบคุมสิทธิ์ตามบทบาท, การจองเข้าออฟฟิศ, ฐานข้อมูลเชิงสัมพันธ์, โพสต์เกรสคิวแอล, เว็บแอปพลิเคชัน

---

* Author's Email: *67130701930@ad.sit.kmutt.ac.th*

[1] นักศึกษาปริญญาโท สาขาวิชาระบบสารสนเทศทางธุรกิจดิจิทัล คณะเทคโนโลยีสารสนเทศ มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี

[2] ผู้ช่วยศาสตราจารย์ คณะเทคโนโลยีสารสนเทศ มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี

Office Space Booking and Access Management System

Krissana Piputpruedtigul*[1] and Suree Funilkul[2]

King Mongkut's University of Technology Thonburi

126 PrachaUthit Rd., BangMod, ThungKru, Bangkok 10140, Thailand

Abstract

This paper presents the development of CoDesk, an office space booking and access management system designed to support more systematic management of office attendance within an organization. The system was developed to address problems arising from the management of booking information through multiple channels, which may result in overlapping bookings, data inconsistencies, difficulties in controlling the number of office users in accordance with each department's capacity, and access to information beyond the scope permitted for each user group. CoDesk was developed as a web application using React and TypeScript for the user interface, ASP.NET for processing the system's application logic, and Supabase PostgreSQL as the primary database for storing relational data and supporting data access control. The developed system supports the creation, modification, and cancellation of booking records, detects and prevents overlapping bookings, provides warnings when booking dates fall on holidays, controls the number of bookings according to the capacity of each department, and determines data visibility based on user roles and departments. In addition, booking data can be summarized in reports for monitoring the number of bookings and the utilization of departmental capacity. As a result, office attendance data can be stored and validated under a consistent set of rules, reducing inconsistencies in data management and supporting the systematic use of information for office space management.

Keywords: capacity control, office booking, PostgreSQL, relational database, role-based access control, web application

---

* Author's Email: *67130701930@ad.sit.kmutt.ac.th*

[1] Postgraduate Student, Information Technology Program, School of Information Technology, King Mongkut's University of Technology Thonburi

[2] Assistant Professor, School of Information Technology, King Mongkut's University of Technology Thonburi

1. บทนำ

ปัจจุบันหลายองค์กรมีการปรับรูปแบบการทำงาน สำนักงานร่วมกัน ส่งผลให้การบริหารจัดการที่นั่งเข้าออฟฟิศกลายเป็นประเด็นสำคัญในการจัดสรรทรัพยากรและการจัดการข้อมูล หากไม่มีระบบที่เหมาะสม อาจเกิดปัญหาการจองซ้ำ การใช้ทรัพยากรไม่สอดคล้องกับความจุของแต่ละฝ่ายและความไม่ชัดเจนในการเข้าถึงข้อมูลของผู้ใช้งานแต่ละกลุ่ม กระบวนการจองที่นั่งแบบเดิมมักอาศัยการบันทึกข้อมูลผ่านเอกสาร ตารางข้อมูลหรือการสื่อสารผ่านเครื่องมือทั่วไปซึ่งไม่ได้ถูกออกแบบมาเพื่อควบคุมกติกาการจองโดยตรงส่งผลให้เกิดปัญหาการจองซ้ำในช่วงเวลาเดียวกัน การตรวจสอบจำนวนผู้ใช้งานในแต่ละฝ่ายทำได้ไม่แม่นยำและข้อมูลที่ได้อาจไม่อยู่ในรูปแบบเดียวกัน ทำให้ติดตาม ตรวจสอบและสรุปผลได้ยาก นอกจากนี้ ยังขาดการกำหนดสิทธิ์การเข้าถึงข้อมูลที่ชัดเจนตามสิทธิ์ของผู้ใช้ เช่น พนักงานทั่วไป ฝ่ายทรัพยากรบุคคล และผู้ดูแลระบบ จึงอาจก่อให้เกิดความเสี่ยงทั้งด้านความถูกต้องของข้อมูลและการเข้าถึงข้อมูลเกินขอบเขต

จากปัญหาดังกล่าวจึงมีแนวคิดในการพัฒนาระบบ **CoDesk** ซึ่งเป็นระบบเว็บสำหรับจองที่นั่งเข้าออฟฟิศเพื่อช่วยให้การจัดการข้อมูลการจองเป็นระบบมากขึ้นลดข้อผิดพลาดจากการบันทึกข้อมูลหลายช่องทาง และสนับสนุนการใช้งานที่สอดคล้องกับสิทธิ์ของผู้ใช้ในองค์กร ระบบนี้มุ่งเน้นการจัดเก็บข้อมูลอย่างเป็นระบบ การกำหนดสิทธิ์การเข้าถึงข้อมูลและการรองรับการใช้งานข้อมูลเพื่อการบริหารจัดการ

โครงการนี้มีวัตถุประสงค์เพื่อพัฒนาระบบจองที่นั่งเข้าออฟฟิศที่สามารถรองรับการใช้งานจริงในองค์กร โดยช่วยลดปัญหาการจองซ้ำ ควบคุมจำนวนผู้จองให้สอดคล้องกับความจุของแต่ละฝ่ายและกำหนดสิทธิ์การใช้งานให้เหมาะสมกับสิทธิ์ของผู้ใช้แต่ละกลุ่ม ขอบเขตของระบบครอบคลุมการจองที่นั่ง ปฏิทินการใช้งาน การจัดการฝ่ายงาน การจัดการพนักงาน การจัดการวันหยุด ระบบรายงานและระบบผู้ใช้กับสิทธิ์การใช้งานโดยกำหนดสิทธิ์หลักของผู้ใช้เป็นพนักงานทั่วไป, ฝ่ายบุคคลและผู้ดูแลระบบ ซึ่งมีขอบเขตการใช้งานแตกต่างกันตามหน้าที่ ทั้งนี้ระบบรองรับการจองทั้งแบบรายวันและแบบช่วงเวลา มีการแจ้งเตือนกรณีจองตรงกับวันหยุดและกำหนดมาตรฐานวันที่พร้อมใช้เขตเวลากรุงเทพเพื่อให้การจัดเก็บและแสดงผลข้อมูลเป็นมาตรฐานเดียวกัน และสามารถนำข้อมูลไปใช้ประโยชน์ในการบริหารจัดการได้ต่อไป

ผลที่คาดว่าจะได้รับจากโครงการนี้ คือองค์กรสามารถบริหารการเข้าใช้ออฟฟิศได้อย่างเป็นระบบมากขึ้น ลดปัญหาการจองซ้ำ ลดความคลาดเคลื่อนของข้อมูลและควบคุมการใช้ทรัพยากรให้สอดคล้องกับความจุของแต่ละฝ่ายได้อย่างชัดเจน ผู้ใช้งานแต่ละRole สามารถปฏิบัติงานได้ตามสิทธิ์ที่กำหนดช่วยลดความเสี่ยงจากการเข้าถึงข้อมูลเกินขอบเขตและเพิ่มความโปร่งใสในการติดตามข้อมูลการใช้งาน

2. ทฤษฎีและหลักการที่เกี่ยวข้อง

2.1 แนวคิดระบบฐานข้อมูลเชิงสัมพันธ์และความถูกต้องของข้อมูล

โครงการ CoDesk อาศัยแนวคิดของระบบฐานข้อมูลเชิงสัมพันธ์ในการจัดเก็บและเชื่อมโยงข้อมูลที่เกี่ยวข้องกับการจองที่นั่งเข้าออฟฟิศ เช่น ข้อมูลผู้ใช้ ฝ่ายงาน การจอง วันหยุด และสิทธิ์การใช้งานโดยแนวคิดนี้ช่วยให้สามารถกำหนดความสัมพันธ์ของข้อมูลได้อย่างชัดเจนลดความซ้ำซ้อนของข้อมูลและรองรับการกำหนดข้อบังคับของข้อมูลเพื่อให้ข้อมูลมีความถูกต้องและตรวจสอบได้ เช่น การกำหนดคีย์หลัก (Primary Key) เพื่อระบุเอกลักษณ์ของข้อมูล การกำหนดคีย์ต่างประเทศ (Foreign Key) เพื่อเชื่อมโยงความสัมพันธ์ระหว่างตารางและการกำหนดข้อบังคับเพิ่มเติม เช่น NOT NULL, UNIQUE และ CHECK เพื่อควบคุมค่าข้อมูลให้เป็นไปตามเงื่อนไขที่ระบบกำหนด [1], [2] สำหรับโครงการนี้ เลือกใช้ Supabase PostgreSQL เป็นฐานข้อมูลหลัก เนื่องจากรองรับการจัดเก็บข้อมูลเชิงสัมพันธ์และสามารถทำงานร่วมกับระบบยืนยันตัวตนและการควบคุมสิทธิ์ใน

สภาพแวดล้อมเดียวกันได้ [2], [4] นอกจากนี้การออกแบบฐานข้อมูลของระบบยังต้องคำนึงถึงกติกาทางธุรกิจที่สำคัญ เช่น การป้องกันการจองซ้ำในช่วงเวลาเดียวกัน การควบคุมจำนวนผู้จองให้สอดคล้องกับความจุของแต่ละฝ่าย และการรองรับการตรวจสอบข้อมูลย้อนหลัง ซึ่งเป็นองค์ประกอบสำคัญที่ทำให้ข้อมูลมีความถูกต้องและพร้อมนำไปใช้ประโยชน์ต่อในการบริหารจัดการ

2.2 การควบคุมสิทธิ์การเข้าถึงข้อมูล

ระบบ CoDesk มีผู้ใช้งานหลายบทบาท ได้แก่ พนักงานทั่วไป, พนักงานฝ่ายบุคคล และผู้ดูแลระบบ ดังนั้นการออกแบบระบบจึงต้องคำนึงถึงการกำหนดสิทธิ์การเข้าถึงข้อมูลตามบทบาทของผู้ใช้ แนวคิดที่นำมาใช้คือ Role-Based Access Control (RBAC) ซึ่งเป็นแนวทางในการกำหนดสิทธิ์ตามหน้าที่ของผู้ใช้แต่ละกลุ่ม [3] ขณะเดียวกันในระดับฐานข้อมูลยังใช้แนวคิด Row-Level Security (RLS) เพื่อควบคุมการเข้าถึงข้อมูลในระดับแถว ทำให้ระบบสามารถจำกัดการมองเห็นและการแก้ไขข้อมูลได้เหมาะสมกับสิทธิ์ของแต่ละสิทธิ์ [4] การใช้ RBAC ร่วมกับ RLS ช่วยให้ระบบมีความปลอดภัยมากขึ้นและลดความเสี่ยงจากการเข้าถึงข้อมูลเกินขอบเขตโดยเฉพาะในระบบที่มีทั้งข้อมูลธุรกรรม ข้อมูลผู้ใช้และข้อมูลเพื่อการรายงาน

2.3 เทคโนโลยีที่ใช้พัฒนาระบบ

ในส่วนของการพัฒนาระบบหน้าเว็บ โครงการนี้ใช้ React สำหรับสร้างส่วนติดต่อผู้ใช้ เนื่องจากเหมาะกับการพัฒนาเว็บแอปพลิเคชันที่มีการโต้ตอบกับผู้ใช้หลายรูปแบบและช่วยจัดการโครงสร้างของหน้าจอในลักษณะ component-based ได้อย่างมีประสิทธิภาพ [5] นอกจากนี้ยังใช้ TypeScript เพื่อช่วยกำหนดชนิดข้อมูลให้ชัดเจนขึ้นในการพัฒนาหน้าจอซึ่งช่วยลดข้อผิดพลาดจากการส่งข้อมูลระหว่างส่วนต่างๆ ของระบบ สำหรับส่วนการประมวลผลฝั่งหลังบ้าน เลือกใช้ ASP.NET เพื่อรองรับตรรกะการทำงานของระบบ การตรวจสอบเงื่อนไขทางธุรกิจ และการเชื่อมต่อกับฐานข้อมูล [6] เมื่อทำงานร่วมกับ Supabase PostgreSQL และ Supabase Auth จึงทำให้ระบบสามารถรองรับทั้งการจัดเก็บข้อมูล การยืนยันตัวตนและการควบคุมสิทธิ์ได้อย่างเหมาะสมกับลักษณะของโครงการ

2.4 การเปรียบเทียบเครื่องมือฐานข้อมูลที่ใช้กับ Oracle

แม้โครงการนี้ไม่ได้ใช้ Oracle Database เป็นฐานข้อมูลหลักแต่ยังคงอาศัยหลักการเดียวกันของระบบฐานข้อมูลเชิงสัมพันธ์ เช่น การจัดเก็บข้อมูลในรูปแบบตาราง การใช้ภาษา SQL การกำหนดความสัมพันธ์ของข้อมูลและการกำหนดข้อบังคับเพื่อรักษาความถูกต้องของข้อมูล [1], [7] ความแตกต่างอยู่ที่ลักษณะของแพลตฟอร์มและบริบทการนำไปใช้งานโดย Oracle มักถูกใช้ในระบบองค์กรขนาดใหญ่ที่มีเครื่องมือและสภาพแวดล้อมเฉพาะทางของตนเอง ขณะที่ Supabase PostgreSQL เหมาะกับโครงการลักษณะเว็บแอปพลิเคชันที่ต้องการความยืดหยุ่นในการพัฒนา การเชื่อมต่อกับระบบยืนยันตัวตนและการใช้งานร่วมกับเครื่องมือพัฒนาสมัยใหม่ [2], [4] ดังนั้นการเลือกใช้Supabase PostgreSQL ในโครงการนี้จึงเหมาะสมกับขอบเขตของระบบ CoDesk และยังคงยึดหลักการสำคัญของงานออกแบบฐานข้อมูลเชิงสัมพันธ์เช่นเดียวกับที่ใช้ใน Oracle

3. วิธีดำเนินการ

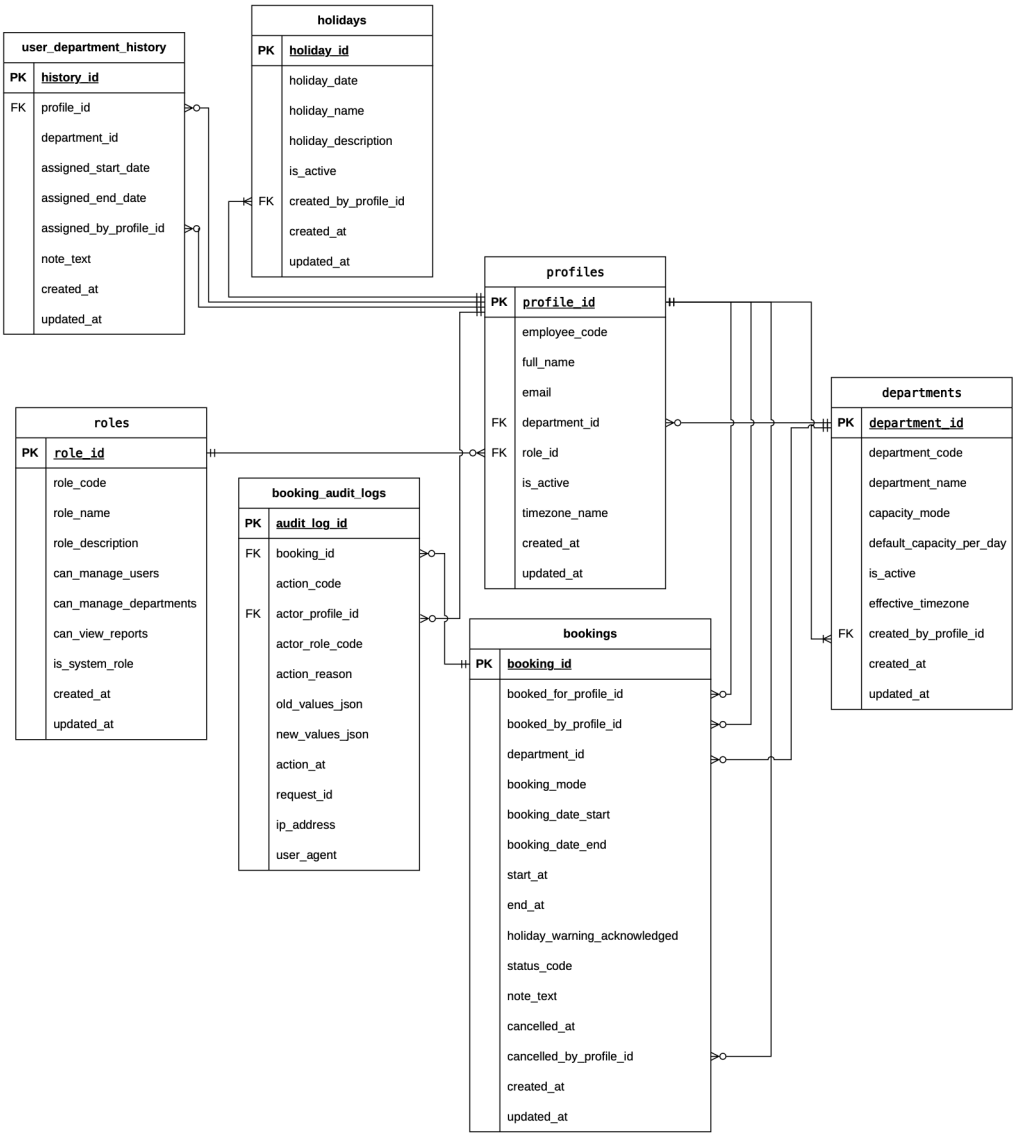
3.1 ความต้องการเชิงระบบ

ระบบ CoDesk ต้องรองรับการจองที่นั่งเข้าออฟฟิศผ่านเว็บแอปพลิเคชันโดยผู้ใช้สามารถจองได้ทั้งแบบรายวันและแบบช่วงวันหรือช่วงเวลา สามารถจองข้ามวัน จองย้อนหลังและจองล่วงหน้าได้ ระบบต้องตรวจสอบไม่ให้ผู้ใช้รายเดิมจองซ้อนในช่วงเวลาเดียวกัน และต้องตรวจสอบจำนวนผู้จองให้สอดคล้องกับความจุของแต่ละฝ่ายตามเงื่อนไขที่กำหนดไว้ นอกจากนี้ระบบต้องรองรับการแก้ไขและยกเลิกการจองตามสิทธิ์ของผู้ใช้ และรองรับการแจ้งเตือนเมื่อวันที่จองตรงกับวันหยุด โดยยังสามารถให้ผู้ใช้ยืนยันการจองต่อได้

ในด้านการใช้งานข้อมูล ระบบต้องแสดงปฏิทินการใช้งานเพื่อให้ผู้ใช้เห็นรายการจองในแต่ละวันได้และต้องกำหนดการมองเห็นข้อมูลตามบทบาทและฝ่ายงาน เช่น employee และhr เห็นข้อมูลของตนเองและคนในฝ่ายเดียวกัน ส่วน admin สามารถเห็นข้อมูลได้ทุกฝ่าย ระบบยังต้องรองรับการจัดการฝ่ายงาน การจัดการพนักงาน การจัดการวันหยุด การจัดการผู้ใช้และสิทธิ์การใช้งาน รวมถึงการสร้างรายงานในหลายมุมมองเพื่อใช้ในการบริหารจัดการ ทั้งนี้ ระบบกำหนดให้ใช้รูปแบบวันที่ YYYY-MM-DD และใช้เขตเวลา Asia/Bangkok เป็นมาตรฐานเดียวกันทั้งระบบ

3.2 การออกแบบโครงสร้างข้อมูลและแผนภาพความสัมพันธ์ข้อมูล

การออกแบบฐานข้อมูลของระบบใช้ schema co_desk เป็นศูนย์กลางโดยกำหนดตารางหลักเพื่อรองรับข้อมูลสำคัญของระบบได้แก่ roles, departments, profiles, holidays, bookings, booking_audit_logs และ user_department_history ซึ่งครอบคลุมทั้งข้อมูลผู้ใช้ ข้อมูลฝ่ายงาน ข้อมูลการจอง ข้อมูลวันหยุด และข้อมูลสำหรับการตรวจสอบย้อนหลัง การกำหนดตารางเหล่านี้ทำให้สามารถแยกหน้าที่ของข้อมูลแต่ละส่วนได้อย่างชัดเจน และช่วยให้รองรับกติกาทางธุรกิจของระบบได้เป็นลำดับมากขึ้น



รูปที่ 3.1 *แผนภาพความสัมพันธ์ข้อมูลของระบบ CoDesk*

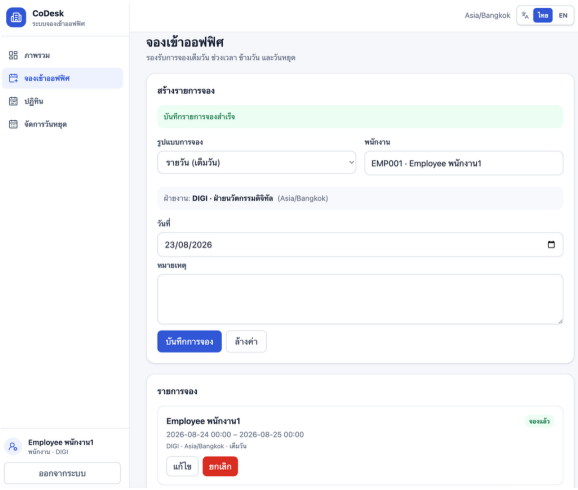
- **roles และ profiles** คือ 1 roles ต้องสามารถถูกกำหนดให้กับ profiles ได้มากกว่า 1 profile โดยแต่ละ profile จะมี role หลักได้เพียง 1 role และในบาง role อาจยังไม่มีผู้ใช้ก็ได้ แต่ profiles ทุก profiles ต้องมี role เดียว เสมอ
- **departments และ profiles** คือ 1 departments สามารถถูกกำหนดให้กับ profiles ได้มากกว่า 1 profile โดยแต่ละ profiles จะสังกัดได้เพียง 1 departments และในบาง departments อาจยังไม่มี profiles แต่ 1 profile ต้องสังกัด department เดียวเสมอ
- **profiles และ bookings (booked_for_profile_id)** คือ 1 profiles สามารถมี bookings ได้มากกว่า 1 bookings โดยแต่ละ bookings จะอ้างอิง profiles ที่ถูกจองได้เพียง 1 profiles และในบาง profiles อาจยังไม่มีรายการจองก็ได้ แต่ทุก bookings ต้องอ้างอิง profiles 1 profiles เสมอ
- **profiles และ bookings (booked_by_profile_id)** คือ 1 profiles สามารถเป็นผู้ทำรายการใน bookings ได้มากกว่า 1 bookings โดยแต่ละ bookings จะอ้างอิงผู้ทำรายการได้เพียง 1 profiles และในบาง profiles อาจยังไม่เคยทำรายการจองก็ได้ แต่ทุก bookings ต้องอ้างอิง profiles 1 profiles เสมอ
- **profiles และ bookings (cancelled_by_profile_id)** คือ 1 profiles สามารถเป็นผู้ยกเลิกรายการใน bookings ได้มากกว่า 1 bookings โดยแต่ละ bookings จะอ้างอิงได้ไม่เกิน 1 profiles และในบาง profiles อาจยังไม่เคยยกเลิกรายการจองก็ได้ อีกทั้ง bookings บาง bookings อาจยังไม่มีผู้ยกเลิกก็ได้ เพราะยังไม่ถูกยกเลิก
- **departments และ bookings** คือ 1 departments สามารถมี bookings ได้มากกว่า 1 bookings โดยแต่ละ bookings จะอ้างอิงได้เพียง 1 departments และในบาง departments อาจยังไม่มีรายการจองก็ได้ แต่ bookings ทุก bookings ต้องอ้างอิง departments เดียวเสมอ
- **bookings และ booking_audit_logs** คือ 1 bookings สามารถมี booking_audit_logs ได้มากกว่า 1 booking_audit_logs โดยแต่ละ booking_audit_logs จะอ้างอิงได้เพียง 1 bookings และในบาง bookings อาจยังไม่มีประวัติการเปลี่ยนแปลง แต่ทุกข้อมูลใน booking_audit_logs ต้องอ้างอิง bookings 1 bookings เสมอ
- **profiles และ booking_audit_logs (actor_profile_id)** คือ 1 profiles สามารถเป็นผู้กระทำรายการใน booking_audit_logs ได้มากกว่า 1 โดยแต่ละ booking_audit_logs จะอ้างอิง profiles ได้เพียง 1 profiles และในบาง profiles อาจยังไม่เคยมีการกระทำที่ถูกบันทึกไว้ก็ได้ แต่ทุก booking_audit_logs ต้องอ้างอิง profiles เดียว เสมอ
- **profiles และ user_department_history** คือ 1 profile สามารถมี user_department_history ได้มากกว่า 1 โดยแต่ละ user_department_history จะอ้างอิงได้เพียง 1 profiles และในบาง profiles อาจยังไม่มีประวัติการสังกัดฝ่ายก็ได้ แต่ทุก user_department_history ต้องอ้างอิง profiles เดียวเสมอ

4. ผลการดำเนินการ

ผลการพัฒนาระบบ CoDesk ในส่วนนี้แสดงการทำงานที่เกี่ยวข้องกับการจองเข้าออฟฟิศ การตรวจสอบเงื่อนไขก่อนบันทึก การควบคุมความจุของฝ่าย การกำหนดสิทธิ์ในการเรียกดูข้อมูล และการสรุปข้อมูลเพื่อจัดทำรายงานโดยแสดงหน้าจอที่เกิดขึ้นจริงควบคู่กับคำสั่ง SQL ที่เกี่ยวข้องเพื่อให้เห็นว่าข้อมูลจากหน้าเว็บถูกตรวจสอบและนำไปใช้กับฐานข้อมูลอย่างไร

4.1 การบันทึกและตรวจสอบรายการจองเข้าออฟฟิศ

จากรูปที่ 4.1 ผู้ใช้งานเลือกรูปแบบการจอง ระบุวันที่และกดปุ่มบันทึกการจอง เมื่อข้อมูลผ่านการตรวจสอบ ระบบจะเพิ่มรายการลงฐานข้อมูล และแสดงข้อความยืนยัน พร้อมรายการจองที่สร้างขึ้น สถานะรายการ และปุ่ม สำหรับแก้ไขหรือยกเลิก



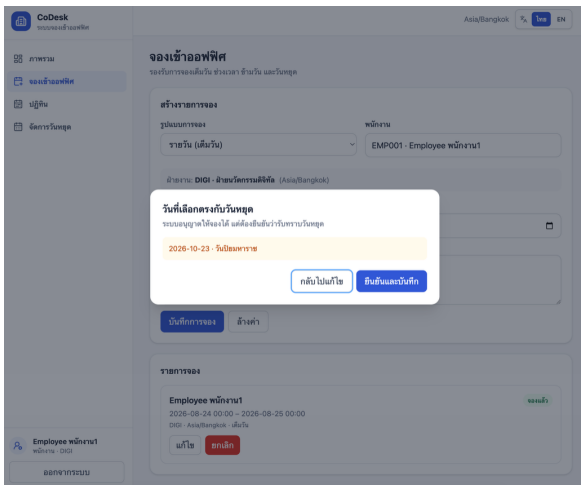
รูปที่ 4.1 การบันทึกรายการจองเข้าออฟฟิศสำเร็จ

คำสั่งสำหรับบันทึกรายการแสดงในรูปที่ 4.2 บรรทัดที่ 262–265 ระบุตาราง co_desk.bookings และคอลัมน์ที่จะจัดเก็บ ได้แก่ พนักงานที่จองเข้าออฟฟิศ ผู้ดำเนินการจอง ฝ่าย รูปแบบการจอง วันที่และเวลา สถานะการ รับทราบ คำเตือนวันหยุด สถานะรายการ และหมายเหตุ บรรทัดที่ 266–269 กำหนดค่าที่นำไปบันทึก โดยกำหนด สถานะเริ่มต้นเป็น booked ส่วนบรรทัดที่ 270 ใช้ RETURNING * INTO v_booking เพื่อรับข้อมูลรายการที่เพิ่มสำเร็จไว้ใช้ในขั้นตอนถัดไปของระบบ

```
262  INSERT INTO co_desk.bookings (
263      booked_for_profile_id, booked_by_profile_id, department_id, booking_mode,
264      booking_date_start, booking_date_end, start_at, end_at,
265      holiday_warning_acknowledged, status_code, note_text
266  ) VALUES (
267      p_booked_for_profile_id, p_booked_by_profile_id, v_department_id,
268      p_booking_mode,v_date_start, v_date_end, v_start_at, v_end_at,
269      p_holiday_warning_acknowledged, 'booked', NULLIF(btrim(p_note_text), '')
270  ) RETURNING * INTO v_booking;
```

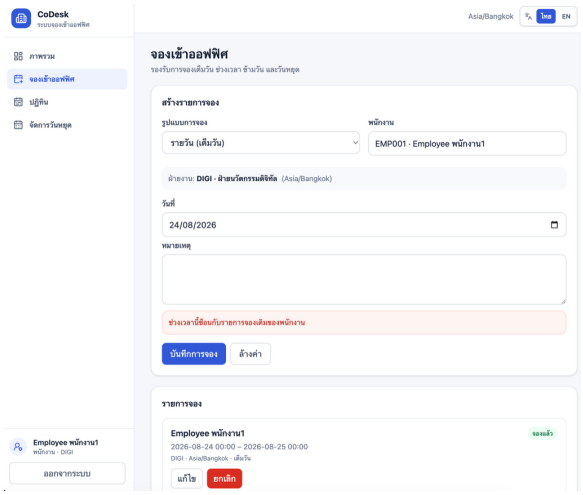
รูปที่ 4.2 คำสั่ง SQL สำหรับบันทึกข้อมูลการจองเข้าออฟฟิศ

ก่อนเพิ่มรายการลงตาราง bookings ระบบจะตรวจสอบวันที่ที่เลือกกับข้อมูลวันหยุด หากพบว่าเป็นวันหยุด ระบบจะยังไม่บันทึกรายการทันที แต่แสดงวันที่และชื่อวันหยุดให้ผู้ใช้งานรับทราบก่อน ดังรูปที่ 4.3 ผู้ใช้งานสามารถกลับไปแก้ไขวันที่ หรือเลือก “ยืนยันและบันทึก” เพื่อดำเนินการจองต่อได้



รูปที่ 4.3 การแจ้งเตือนเมื่อวันที่จองตรงกับวันหยุด

ระบบยังตรวจสอบรายการเดิมของพนักงานก่อนบันทึก หากช่วงเวลาที่เลือกซ้อนทับกับรายการที่มีอยู่ ระบบจะไม่เพิ่มรายการใหม่และแสดงข้อความแจ้งเตือน ดังรูปที่ 4.4



รูปที่ 4.4 การป้องกันการจองซ้ำของพนักงาน

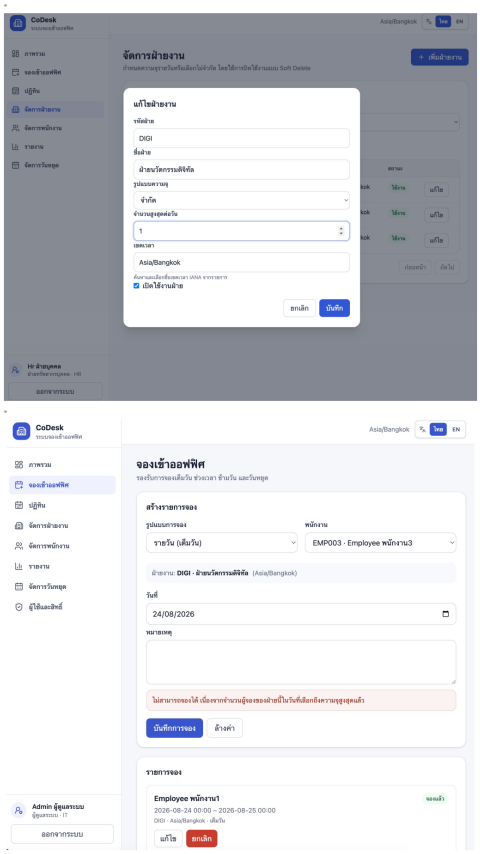
รูปที่ 4.5 แสดงคำสั่งที่ใช้ตรวจสอบการจองซ้ำ บรรทัดที่ 34–43 สร้างค่า hasConflict โดยค้นหาข้อมูลจากตาราง co_desk.bookings ของพนักงานคนเดียวกัน เฉพาะรายการที่มีสถานะ booked และไม่นับรายการที่กำหนดให้ยกเว้น เงื่อนไขในบรรทัดที่ 41–42 ตรวจว่าช่วงเวลาของรายการใหม่ซ้อนกับรายการเดิมหรือไม่ ส่วนบรรทัดที่ 44–54 ใช้ค้นหา booking_id ของรายการที่ซ้อนกันรายการแรกและส่งกลับเป็น conflictingBookingId

```sql
SELECT jsonb_build_object(
    'hasConflict', EXISTS (
        SELECT 1
        FROM co_desk.bookings b
        WHERE b.booked_for_profile_id = p_booked_for_profile_id
          AND b.status_code = 'booked'
          AND b.booking_id IS DISTINCT FROM p_exclude_booking_id
          AND p_start_at < b.end_at
          AND p_end_at > b.start_at
    ),
    'conflictingBookingId', (
        SELECT b.booking_id
        FROM co_desk.bookings b
        WHERE b.booked_for_profile_id = p_booked_for_profile_id
          AND b.status_code = 'booked'
          AND b.booking_id IS DISTINCT FROM p_exclude_booking_id
          AND p_start_at < b.end_at
          AND p_end_at > b.start_at
        ORDER BY b.start_at
        LIMIT 1
    )
);
```

รูปที่ 4.5 คำสั่ง SQL สำหรับตรวจสอบการจองซ้ำ

4.2 การควบคุมจำนวนผู้จองตามความจุของฝ่าย

การทดสอบความจุของฝ่ายกำหนดฝ่าย DIGI เป็นรูปแบบจำกัดและกำหนดจำนวนสูงสุดต่อวันเท่ากับ 1 คน ดังรูปที่ 4.6 (ก) เมื่อวันดังกล่าวมีผู้จองครบแล้ว และมีการพยายามเพิ่มรายการอีก ระบบจะแสดงข้อความว่าไม่สามารถจองได้เนื่องจากจำนวนผู้จองถึงความจุสูงสุดของฝ่าย ดังรูปที่ 4.6 (ข)



รูปที่ 4.6 การกำหนดและควบคุมความจุของฝ่าย DIGI (ก) กำหนดความจุ (ข) ผลเมื่อความจุเต็ม

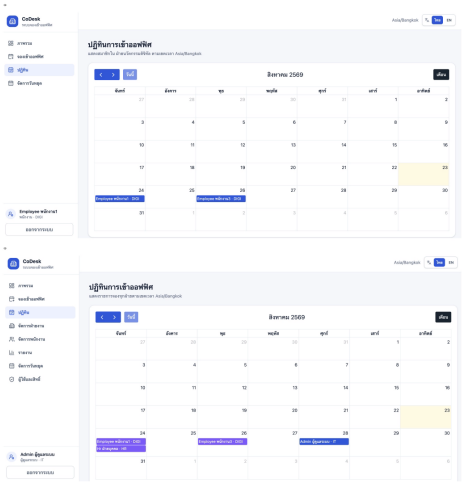
คำสั่งตรวจสอบความจุแสดงในรูปที่ 4.7 บรรทัดที่ 74–87 ใช้ COUNT นับรายการจองที่มีสถานะ booked ของฝ่ายในแต่ละวันเป็น current_count ส่วนบรรทัดที่ 88–96 นำค่าดังกล่าวมาคำนวณ projected_count จาก current_count + 1 และตรวจว่า capacity_mode เป็น limited พร้อมกับมีการตรวจสอบด้วยว่า projected_count มากกว่า default_capacity_per_day หรือไม่ หากเงื่อนไขเป็นจริง ค่า exceeded จะเป็นจริงและระบบจะไม่เพิ่มรายการจองใหม่

```sql
daily AS (
    SELECT d.business_date,
           COUNT(b.booking_id)::integer AS current_count
    FROM department
    CROSS JOIN LATERAL co_desk.booking_touched_dates(
        p_start_at, p_end_at, department.effective_timezone
    ) d
    LEFT JOIN co_desk.bookings b
      ON b.department_id = p_department_id
     AND b.status_code = 'booked'
     AND b.booking_id IS DISTINCT FROM p_exclude_booking_id
     AND b.booking_date_start <= d.business_date
     AND b.booking_date_end >= d.business_date
    GROUP BY d.business_date
), result AS (
    SELECT daily.business_date,
           department.capacity_mode,
           department.default_capacity_per_day,
           daily.current_count,
           daily.current_count + 1 AS projected_count,
           department.capacity_mode = 'limited'
               AND daily.current_count + 1 > department.default_capacity_per_day AS exceeded
    FROM daily CROSS JOIN department)
```

รูปที่ 4.7 คำสั่ง SQL สำหรับตรวจสอบความจุของฝ่าย

4.3 การแสดงข้อมูลตามบทบาทและฝ่ายงานของผู้ใช้งาน

ผลการทดสอบสิทธิ์การเรียกดูข้อมูลแสดงในรูปที่ 4.8 เมื่อเข้าสู่ระบบด้วยบทบาท Employee ของฝ่าย DIGI ปฏิทินจะแสดงรายการภายในฝ่าย DIGI ส่วนบทบาท Admin สามารถเห็นรายการจากทุกฝ่าย เช่น DIGI, HR และ IT ในช่วงเวลาเดียวกัน



รูปที่ 4.8 ปฏิทินการเข้าออฟฟิศตามบทบาทผู้ใช้งาน (ก) Employee (ข) Admin

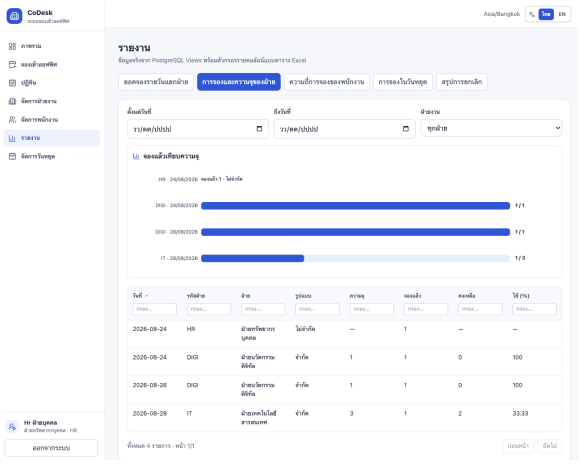
การควบคุมในระดับฐานข้อมูลใช้ Row-Level Security (RLS) ตามรูปที่ 4.9 บรรทัดที่ 100 สร้าง Policy bookings_read สำหรับการ SELECT จากตาราง co_desk.bookings บรรทัดที่ 101 อนุญาตให้ Admin เรียกดูข้อมูลได้ บรรทัดที่ 102 อนุญาตให้ผู้ใช้งานเรียกดูรายการของตนเองและบรรทัดที่ 103 อนุญาตให้เรียกดูรายการในฝ่ายเดียวกัน เงื่อนไขทั้งสามเชื่อมด้วย OR จึงอนุญาตการอ่านข้อมูลเมื่อเข้าเงื่อนไขอย่างใดอย่างหนึ่ง

```
100  CREATE POLICY bookings_read ON co_desk.bookings FOR SELECT USING (
101      co_desk.is_admin()
102      OR booked_for_profile_id = co_desk.current_profile_id()
103      OR department_id = co_desk.current_department_id()
104  );
```

รูปที่ 4.9 คำสั่ง SQL สำหรับกำหนดสิทธิ์การเข้าถึงข้อมูลการจอง

4.4 การนำข้อมูลไปใช้เพื่อการบริหารจัดการ

รูปที่ 4.10 แสดงรายงานการจองและความจุของฝ่าย ซึ่งสรุปข้อมูลทั้งในรูปกราฟและตาราง ประกอบด้วยวันที่ รหัสฝ่าย ชื่อฝ่าย รูปแบบความจุ ความจุสูงสุด จำนวนผู้จอง จำนวนคงเหลือ และอัตราการใช้ความจุเป็นร้อยละทำให้สามารถเปรียบเทียบการใช้ความจุของแต่ละฝ่ายในช่วงเวลาเดียวกันได้



รูปที่ 4.10 รายงานการจองและความจุของฝ่าย

ข้อมูลรายงานถูกจัดเตรียมจาก View vw_department_capacity_utilization ดังรูปที่ 4.11 บรรทัดที่ 20–28 กำหนด View และเลือกข้อมูลวันที่ ฝ่าย ความจุ และจำนวนผู้จอง บรรทัดที่ 29–31 ใช้ CASE คำนวณ remaining_capacity สำหรับฝ่ายแบบ limited และบรรทัดที่ 32–34 คำนวณ utilization_percentage เป็นร้อยละ ส่วนบรรทัดที่ 35–36 นำข้อมูลการจองรายวันจาก vw_daily_department_bookings มา JOIN กับตาราง departments เพื่อให้ได้ข้อมูลฝ่ายที่ใช้ในการแสดงรายงาน

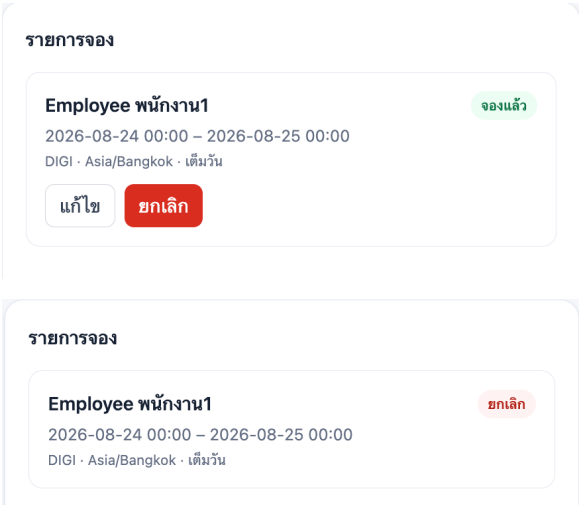
```
20  CREATE OR REPLACE VIEW co_desk.vw_department_capacity_utilization AS
21  SELECT
22      daily.business_date,
23      dep.department_id,
24      dep.department_code,
25      dep.department_name,
26      dep.capacity_mode,
27      dep.default_capacity_per_day AS capacity_per_day,
28      daily.active_booking_total AS booked_count,
29      CASE WHEN dep.capacity_mode = 'limited'
30           THEN GREATEST(dep.default_capacity_per_day - daily.active_booking_total, 0)
31           ELSE NULL END AS remaining_capacity,
32      CASE WHEN dep.capacity_mode = 'limited'
33           THEN ROUND((daily.active_booking_total::numeric / dep.default_capacity_per_day) * 100, 2)
34           ELSE NULL END AS utilization_percentage
35  FROM co_desk.vw_daily_department_bookings daily
36  JOIN co_desk.departments dep ON dep.department_id = daily.department_id;
```

รูปที่ 4.11 คำสั่ง SQL
สำหรับจัดเตรียมข้อมูลรายงานการใช้ความจุของฝ่าย

จากข้อมูลตัวอย่าง ฝ่าย DIGI มีความจุ 1 คนและมีผู้จอง 1 คน จึงมีอัตราการใช้ความจุร้อยละ 100 ส่วนฝ่าย IT มีความจุ 3 คนและมีผู้จอง 1 คน คิดเป็นร้อยละ 33.33 ทำให้เห็นทั้งจำนวนที่ใช้งานแล้ว และจำนวนคงเหลือจากรายงานชุดเดียวกัน

4.5 การยกเลิกรายการจองและบันทึกประวัติการทำรายการ

การยกเลิกรายการทดสอบด้วยบทบาท Employee โดยเลือกรายการที่มีสถานะ “จองแล้ว” และกดปุ่มยกเลิก ดังรูปที่ 4.12 (ก) หลังยืนยันการยกเลิกรายการเดิมเปลี่ยนสถานะเป็น “ยกเลิก” และไม่แสดงปุ่มแก้ไขหรือยกเลิกสำหรับรายการดังกล่าว ดังรูปที่ 4.12 (ข)



รูปที่ 4.12 การยกเลิกรายการจอง (ก) ก่อนยกเลิก (ข) หลังยกเลิก

คำสั่งที่เกี่ยวข้องแสดงในรูปที่ 4.13 บรรทัดที่ 438–443 ใช้ UPDATE ตาราง co_desk.bookings

โดยเปลี่ยน status_code เป็น cancelled บันทึกเวลายกเลิกใน cancelled_at และผู้ที่ดำเนินการใน cancelled_by_profile_id จากนั้นบรรทัดที่ 445–456 ใช้ INSERT INTO ตาราง co_desk.booking_audit_logs เพื่อบันทึกประวัติการยกเลิก โดยจัดเก็บรหัสรายการ ผู้กระทำ บทบาท เหตุผล ค่าเดิม ค่าใหม่ และข้อมูลประกอบสำหรับการตรวจสอบย้อนหลัง ส่วนบรรทัดที่ 457 ส่งผลการยกเลิกกลับไปใช้ในขั้นตอน ถัดไปของระบบ

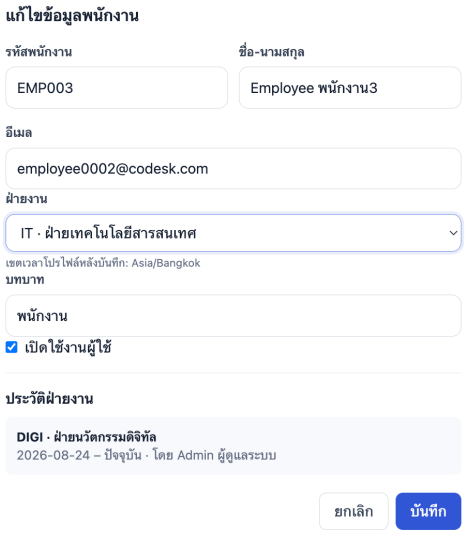
```
438  UPDATE co_desk.bookings SET
439      status_code = 'cancelled',
440      cancelled_at = now(),
441      cancelled_by_profile_id = p_actor_profile_id
442  WHERE booking_id = p_booking_id
443  RETURNING * INTO v_new;
444
445  INSERT INTO co_desk.booking_audit_logs (
446      booking_id, action_code, actor_profile_id, actor_role_code, action_reason,
447      old_values_json, new_values_json, request_id, ip_address, user_agent
448  ) VALUES (
449      p_booking_id, 'cancel', p_actor_profile_id, v_actor_role,
450      COALESCE(NULLIF(btrim(p_action_reason), ''), 'Booking cancelled'),
451      to_jsonb(v_old), to_jsonb(v_new) || jsonb_build_object(
452          'business_timezone', v_timezone,
453          'cancellation_business_date',
454          (v_new.cancelled_at AT TIME ZONE v_timezone)::date
455      ), p_request_id, p_ip_address, left(p_user_agent, 1000)
456  );
457  RETURN jsonb_build_object('success', true, 'code', 'cancelled', 'booking', to_jsonb(v_new));
```

รูปที่ 4.13 คำสั่ง SQL

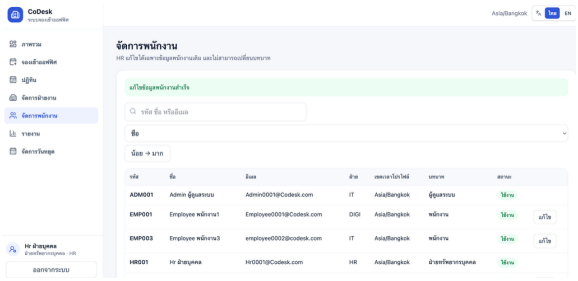
สำหรับยกเลิกรายการจองและบันทึกประวัติการทำรายการ

4.6 การแก้ไขฝ่ายงานและจัดเก็บประวัติพนักงาน

ส่วนจัดการพนักงานทดสอบด้วยบทบาท HR โดยเลือกพนักงาน EMP003 ซึ่งเดิมสังกัดฝ่าย DIGI จากนั้นเปลี่ยนฝ่ายเป็น IT และกดบันทึก ดังรูปที่ 4.14 (ก) หลังบันทึก ระบบแสดงข้อความว่าแก้ไขข้อมูลพนักงานสำเร็จ และรายการ EMP003 ในหน้าจัดการพนักงานแสดงฝ่ายเป็น IT ดังรูปที่ 4.14 (ข)



รูปที่ 4.14 การเปลี่ยนฝ่ายงานของพนักงาน EMP003 (ก) ก่อนบันทึก



รูปที่ 4.14 การเปลี่ยนฝ่ายงานของพนักงาน EMP003 (ข) หลังบันทึกสำเร็จ

การแก้ไขข้อมูลพนักงานใช้คำสั่งในรูปที่ 4.15 บรรทัดที่ 241–247 ใช้ UPDATE ตาราง co_desk.profiles เพื่อปรับรหัสพนักงาน ชื่อ อีเมล department_id, role_id และสถานะ is_active ตามค่าที่ได้รับจากหน้าจอ ส่วนบรรทัดที่ 248 ใช้ WHERE profile_id = @profile_id เพื่อระบุพนักงานที่ต้องการแก้ไข

```
241  UPDATE co_desk.profiles SET
242      employee_code = upper(@employee_code),
243      full_name = @full_name,
244      email = lower(@email),
245      department_id = @department_id,
246      role_id = @role_id,
247      is_active = @is_active
248  WHERE profile_id = @profile_id
```

รูปที่ 4.15 คำสั่ง SQL สำหรับแก้ไขข้อมูลพนักงานและฝ่ายงาน

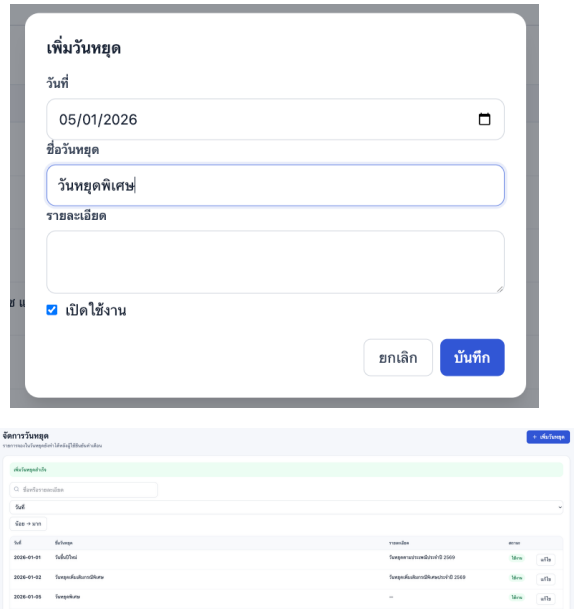
เมื่อ department_id เปลี่ยน ระบบจะบันทึกประวัติฝ่ายงานตามรูปที่ 4.16 โดยบรรทัดที่ 150 ตรวจว่าฝ่ายใหม่แตกต่างจากฝ่ายเดิมหรือไม่ ส่วนบรรทัดที่ 151–155 ใช้ในการ UPDATE ตาราง co_desk.user_department_history เพื่อกำหนด assigned_end_date ให้ประวัติฝ่ายเดิม จากนั้นบรรทัดที่ 157–161 ใช้ INSERT INTO ตารางเดียวกัน เพื่อเพิ่มประวัติฝ่ายใหม่ พร้อมวันที่เริ่มสังกัดและผู้ดำเนินการทำให้สามารถตรวจสอบการเปลี่ยนฝ่ายของพนักงานย้อนหลังได้

```
150  ELSIF NEW.department_id IS DISTINCT FROM OLD.department_id THEN
151      UPDATE co_desk.user_department_history
152      SET assigned_end_date = GREATEST(v_business_date, assigned_start_date),
153          note_text = COALESCE(note_text, 'Department assignment closed')
154      WHERE profile_id = NEW.profile_id
155        AND assigned_end_date IS NULL;
156
157      INSERT INTO co_desk.user_department_history (
158          profile_id, department_id, assigned_start_date, assigned_by_profile_id, note_text
159      ) VALUES (
160          NEW.profile_id, NEW.department_id, v_business_date, v_actor, 'Department changed'
161      );
```

รูปที่ 4.16 คำสั่ง SQL สำหรับบันทึกประวัติการเปลี่ยนฝ่ายของพนักงาน

4.7 การจัดการข้อมูลวันหยุด

การเพิ่มข้อมูลวันหยุดทดสอบด้วยบทบาท HR โดยเปิดเมนูจัดการวันหยุดและระบุวันที่ 05/01/2026 ชื่อ “วันหยุดพิเศษ” พร้อมกำหนดสถานะเปิดใช้งาน ดังรูปที่ 4.17 (ก) เมื่อกดบันทึก ระบบแสดงข้อความ “เพิ่มวันหยุดสำเร็จ” และรายการวันที่ 2026-01-05 ปรากฏในตารางวันหยุด ดังรูปที่ 4.17 (ข)



รูปที่ 4.17 การเพิ่มข้อมูลวันหยุด (ก) ก่อนบันทึก (ข) หลังบันทึกสำเร็จ

คำสั่งเพิ่มข้อมูลวันหยุดแสดงในรูปที่ 4.18 บรรทัดที่ 344 ระบุตาราง co_desk.holidays และคอลัมน์ที่จะบันทึก ได้แก่ วันที่ ชื่อวันหยุด รายละเอียด สถานะ และผู้สร้างรายการ บรรทัดที่ 345 กำหนดค่าที่รับจากหน้าจอ ส่วนบรรทัดที่ 346–347 ใช้ RETURNING เพื่อรับข้อมูลวันหยุดที่เพิ่มสำเร็จกลับไปใช้ในขั้นตอนถัดไปของระบบ

```
344  INSERT INTO co_desk.holidays (holiday_date, holiday_name, holiday_description, is_active, created_by_profile_id)
345  VALUES (@date, @name, @description, @is_active, @actor_id)
346  RETURNING holiday_id, holiday_date, holiday_name, holiday_description, is_active,
347            created_by_profile_id, created_at, updated_at
```

รูปที่ 4.18 คำสั่ง SQL สำหรับเพิ่มข้อมูลวันหยุด

5. บทสรุป

5.1 สรุปผลการดำเนินงาน

โครงการ CoDesk พัฒนาขึ้นเพื่อช่วยแก้ปัญหาการจัดการข้อมูลการเข้าใช้ออฟฟิศที่อาจเกิดจากการดำเนินงานผ่านหลายช่องทาง เช่น การจองซ้ำในช่วงเวลาเดียวกัน การควบคุมจำนวนผู้เข้าใช้งานให้สอดคล้องกับความจุของแต่ละฝ่าย และการเข้าถึงข้อมูลเกินขอบเขตของผู้ใช้งานแต่ละกลุ่ม โดยพัฒนาเป็น Web Application และจัดเก็บข้อมูลในฐานข้อมูล Supabase PostgreSQL เพื่อให้ข้อมูลการจองและข้อมูลที่เกี่ยวข้องถูกจัดเก็บและตรวจสอบภายใต้โครงสร้างเดียวกัน

จากผลการดำเนินงานพบว่า ระบบสามารถตรวจสอบและป้องกันการจองซ้ำ แจ้งเตือนเมื่อวันที่จองตรงกับวันหยุด ควบคุมจำนวนผู้จองตามความจุที่กำหนดของแต่ละฝ่าย และกำหนดการมองเห็นข้อมูลตามบทบาทและฝ่ายงานของผู้ใช้งานได้ นอกจากนี้ ข้อมูลการจองยังสามารถนำมาสรุปเป็นรายงานเพื่อให้ฝ่ายที่เกี่ยวข้องตรวจสอบจำนวนผู้จองและการใช้ความจุของแต่ละฝ่ายได้ ผลที่ได้ช่วยลดความคลาดเคลื่อนในการจัดการข้อมูล เพิ่มความชัดเจนในการควบคุมการเข้าใช้ออฟฟิศ และทำให้ข้อมูลสามารถนำไปใช้ประกอบการบริหารจัดการได้อย่างเป็นระบบมากขึ้น

5.2 แนวทางการพัฒนาในอนาคต

แนวทางสำคัญในการพัฒนาระบบต่อไป คือ การเชื่อมต่อการเข้าสู่ระบบของ CoDesk กับ Microsoft Entra ID (เดิม Azure Active Directory: Azure AD) ขององค์กร เพื่อให้พนักงานสามารถใช้บัญชี Microsoft ของบริษัทเข้าสู่ระบบในลักษณะ Single Sign-On (SSO) แทนการจัดการบัญชีและรหัสผ่านแยกเฉพาะสำหรับ CoDesk ซึ่งจะช่วยลดการจัดการบัญชีผู้ใช้งานซ้ำ และทำให้กระบวนการยืนยันตัวตนสอดคล้องกับระบบขององค์กร โดย Microsoft Entra ID จะทำหน้าที่ยืนยันตัวตนผู้ใช้งาน (Authentication) ขณะที่ CoDesk ยังคงกำหนดบทบาทฝ่ายงาน และขอบเขตการเข้าถึงข้อมูลของ Employee, HR และ Admin ภายในระบบ (Authorization) ตามโครงสร้างที่พัฒนาไว้เดิม

6. เอกสารอ้างอิง

[1] PostgreSQL Global Development Group, “PostgreSQL Documentation,” PostgreSQL. [Online]. Available: https://www.postgresql.org/docs/current/. [Accessed: Mar. 21, 2026].

[2] Supabase, “Database Overview,” Supabase Docs. [Online]. Available: https://supabase.com/docs/guides/database/overview. [Accessed: Mar. 21, 2026].

[3] National Institute of Standards and Technology, “The NIST Model for Role-Based Access Control: Toward a Unified Standard,” NIST. [Online]. Available: https://www.nist.gov/publications/nist-model-role-based-access-control-towards-unified-standard. [Accessed: Mar. 21, 2026].

[4] Supabase, “Row Level Security,” Supabase Docs. [Online]. Available: https://supabase.com/docs/guides/database/postgres/row-level-security. [Accessed: Mar. 21, 2026].

[5] Meta Platforms, Inc., “Learn React,” React Docs. [Online]. Available: https://react.dev/learn. [Accessed: Mar. 21, 2026].

[6] Microsoft, “Create web APIs with ASP.NET Core,” Microsoft Learn. [Online]. Available: https://learn.microsoft.com/en-us/aspnet/core/web-api/. [Accessed: Mar. 21, 2026].

[7] Oracle, “Oracle Database SQL Language Reference,” Oracle Help Center. [Online]. Available: https://docs.oracle.com/en/database/oracle/oracle-database/23/sqlrf/. [Accessed: Mar. 21, 2026].